

Tucker ALS for Functional Approximation in Chebyshev Bases

May 19, 2026

1 Problem Setup

Given a smooth function $f : \mathbb{R}^3 \rightarrow \mathbb{R}$ sampled on a tensor grid, we approximate it using a Tucker representation built directly from Chebyshev basis matrices.

Let

$$T_x \in \mathbb{R}^{N_x \times (d_x+1)}, \quad T_y \in \mathbb{R}^{N_y \times (d_y+1)}, \quad T_z \in \mathbb{R}^{N_z \times (d_z+1)}$$

be Chebyshev Vandermonde matrices evaluated at the grid points.

We seek the approximation

$$F \approx (T_x A_x, T_y A_y, T_z A_z, G),$$

where $A_x \in \mathbb{R}^{(d_x+1) \times R_x}$, $A_y \in \mathbb{R}^{(d_y+1) \times R_y}$, $A_z \in \mathbb{R}^{(d_z+1) \times R_z}$ are factor matrices, and $G \in \mathbb{R}^{R_x \times R_y \times R_z}$ is the core tensor.

2 ALS Algorithm

The ALS loop alternates updates over the three modes, each optimizing one factor matrix and the core tensor while fixing the others.

Define

$$U_x = T_x A_x, \quad U_y = T_y A_y, \quad U_z = T_z A_z.$$

2.1 Mode-1 Update

We rewrite F with the x -mode singled out via mode-1 unfolding:

$$F_{(1)} \approx U_x G_{(1)} (U_z \otimes U_y)^T,$$

where $F_{(1)} \in \mathbb{R}^{N_x \times (N_y N_z)}$.

Let $YZ = U_z \otimes U_y \in \mathbb{R}^{(N_y N_z) \times (R_y R_z)}$. We solve

$$\min_{\tilde{G}_x} \|F_{(1)} - T_x \tilde{G}_x YZ^T\|_F.$$

Vectorizing and solving the least-squares problem:

$$\text{vec}(\tilde{G}_x) = (YZ \otimes T_x)^\dagger \text{vec}(F_{(1)}),$$

yielding $\tilde{G}_x \in \mathbb{R}^{(d_x+1) \times (R_y R_z)}$.

Apply SVD and truncate:

$$\tilde{G}_x = U_x^c S_x V_x^T, \quad A_x = U_x^c(:, 1 : R_x), \quad G_{(1)} = S_x(1 : R_x, 1 : R_x) V_x(1 : R_x, :).$$

Reshape $G_{(1)}$ back into the core tensor G .

2.2 Mode-2 and Mode-3 Updates

The mode-2 and mode-3 updates proceed analogously, working with the respective mode unfoldings $F_{(2)}$ and $F_{(3)}$, and updating A_y , A_z , and the core G accordingly. Iterate through all three modes until convergence.

3 Extension to Higher Dimensions

For a d -dimensional function with Chebyshev Vandermonde matrices

$$T_k \in \mathbb{R}^{N_k \times (m_k + 1)}, \quad k = 1, \dots, d,$$

we seek

$$F \approx (T_1 A_1, T_2 A_2, \dots, T_d A_d, G),$$

with $A_k \in \mathbb{R}^{(m_k + 1) \times R_k}$ and $G \in \mathbb{R}^{R_1 \times \dots \times R_d}$.

The mode- n update solves

$$\text{vec}(F_{(n)}) = (U_{\neg n} \otimes T_n) \text{vec}(\tilde{G}_n),$$

where $U_{\neg n} = U_d \otimes \dots \otimes U_{n+1} \otimes U_{n-1} \otimes \dots \otimes U_1$ and $U_k = T_k A_k$.

SVD and truncation yield $A_n = U_n^c(:, 1 : R_n)$ and an updated core G . Cycle through $n = 1, \dots, d$ until convergence.

4 Numerical Experiments

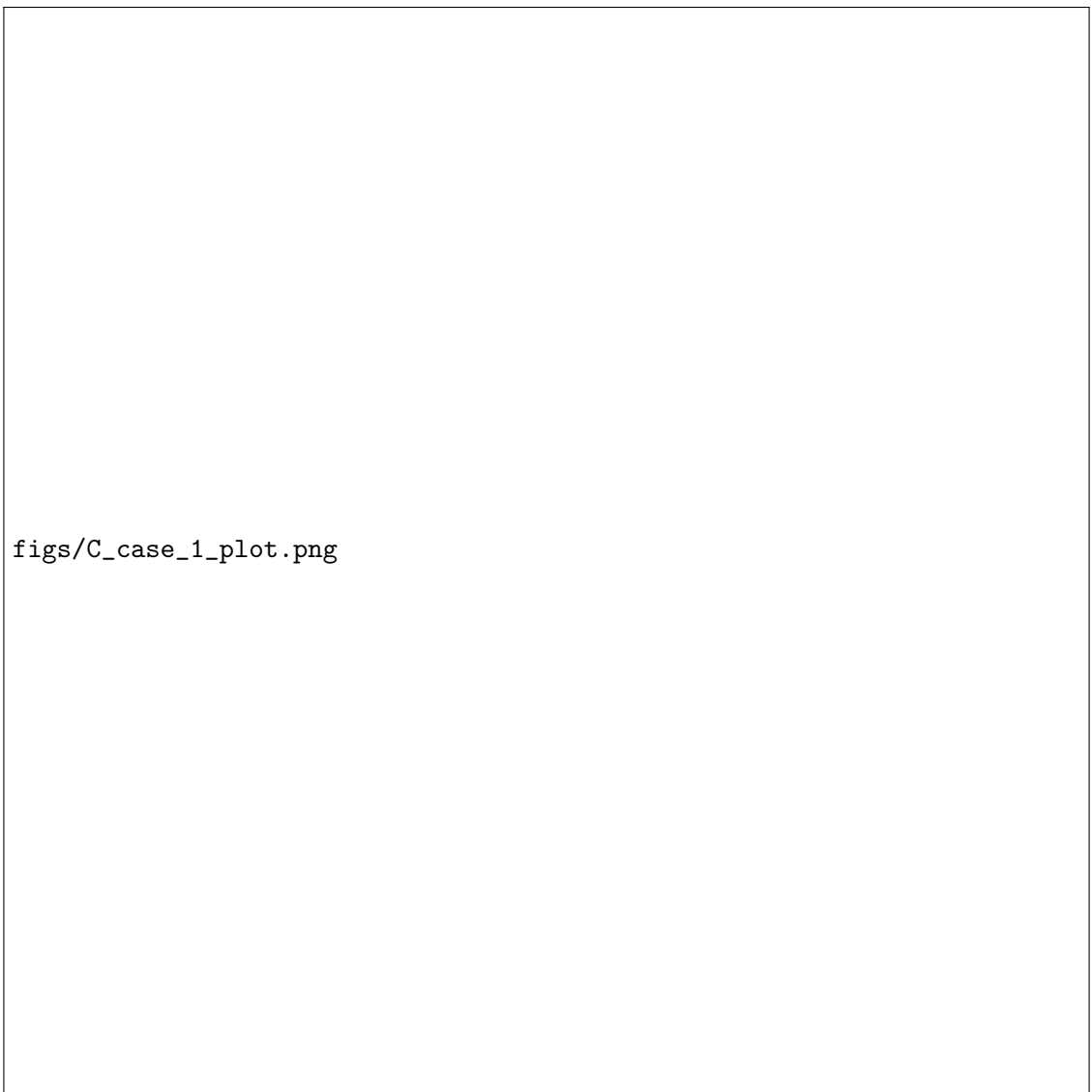
We test the algorithm on 2D anisotropic functions of the form

$$f(x, y) = \frac{1}{1 + \|C[x, y]^T\|^2}, \quad (x, y) \in [-1, 1]^2,$$

where $C \in \mathbb{R}^{2 \times 2}$ controls the anisotropy and orientation.

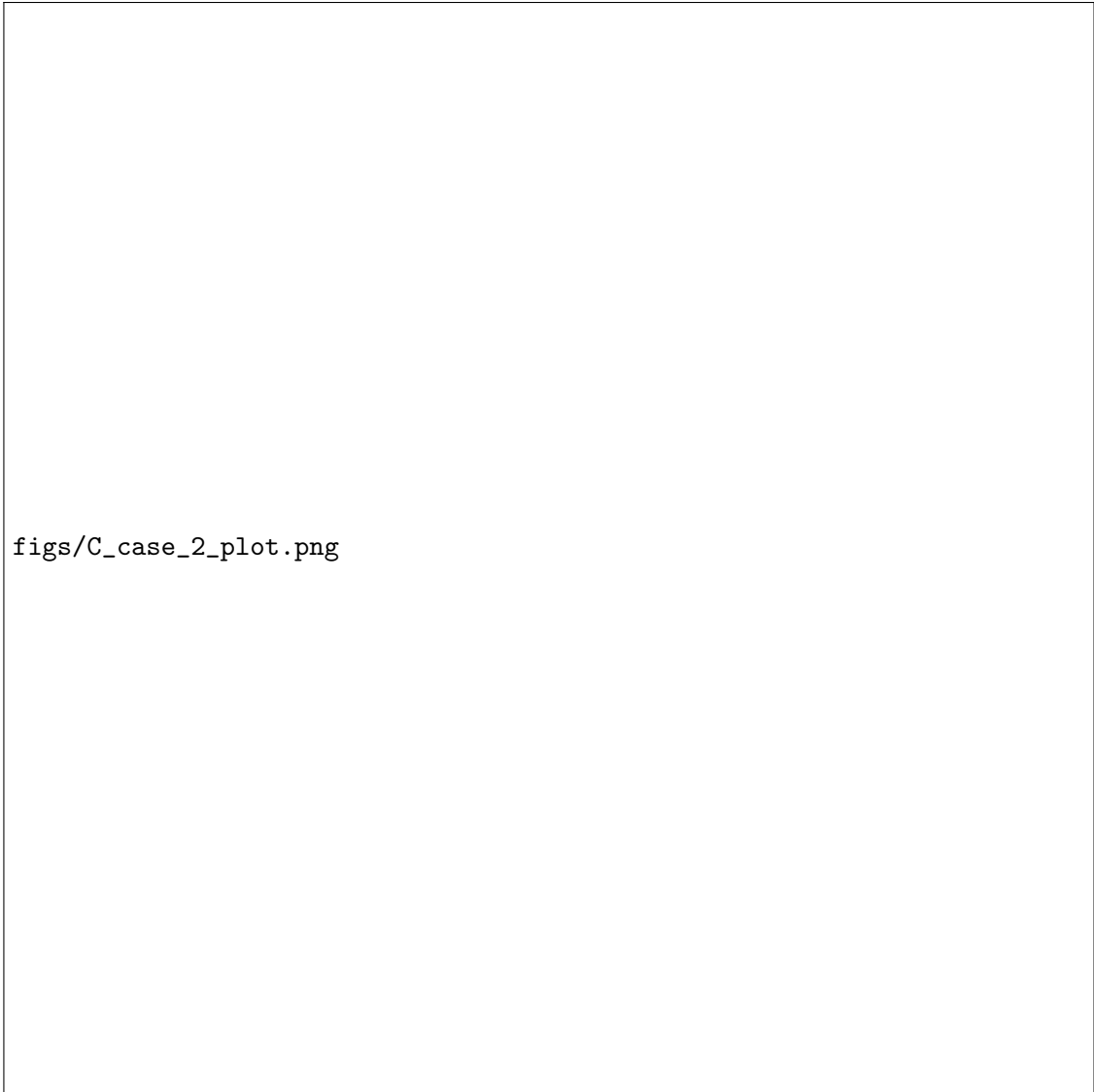
Case 1: $C = \begin{bmatrix} 5 & 0 \\ 0 & 1 \end{bmatrix}$ (horizontal stretching).

Case 2: $C = \begin{bmatrix} 1 & 0 \\ 0 & 5 \end{bmatrix} R(30^\circ)$ (rotated vertical stretching).



figs/C_case_1_plot.png

Figure 1: Case 1 test function.



figs/C_case_2_plot.png

Figure 2: Case 2 test function.

4.1 Approximation Quality

We measure the ℓ^2 coefficient error between the Chebyshev coefficient tensor reconstructed from each method's Tucker approximation at a given rank and the best degree- d Chebyshev interpolant. Comparisons include:

ALS Tucker: The direct ALS algorithm described above.

Tucker Baseline: Two-stage approach: compute full Chebyshev interpolant, then apply Tucker decomposition.

ADAM/BFGS: Gradient-based optimization methods.

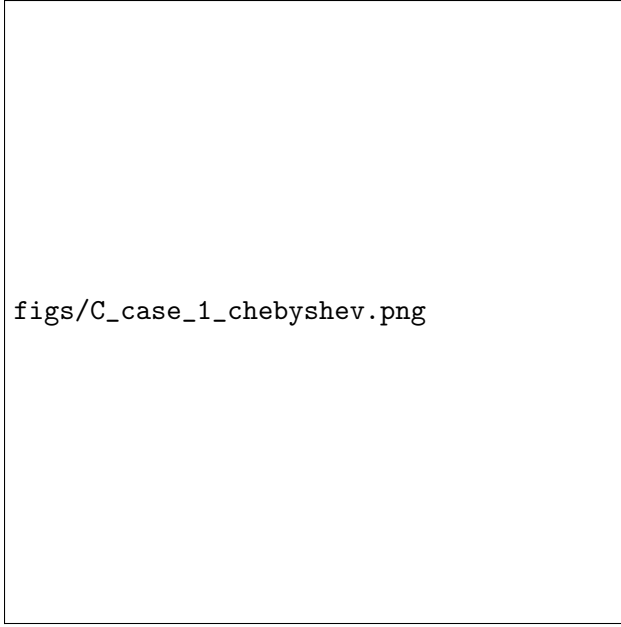


Figure 3: Case 1: Coefficient error vs. Tucker rank for degrees $d = 16, 32, 64$ on Chebyshev grids.

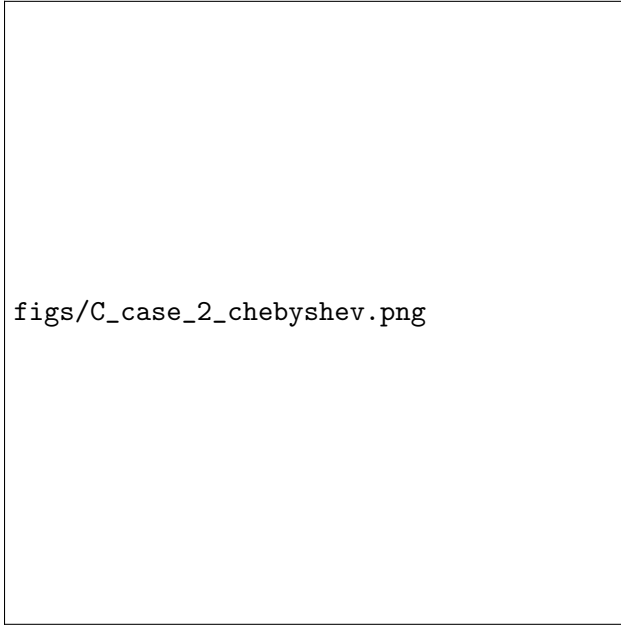


Figure 4: Case 2: Coefficient error vs. Tucker rank for degrees $d = 16, 32, 64$ on Chebyshev grids.



Figure 5: Case 1: Coefficient error using 320×320 random sampling points.



Figure 6: Case 2: Coefficient error using 320×320 random sampling points.

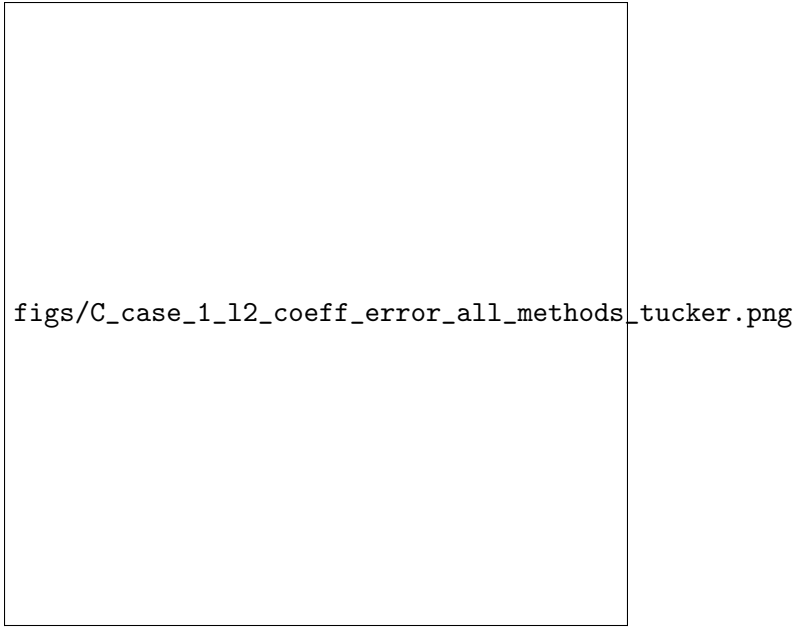


Figure 7: Case 1: Comparison of ALS Tucker, Tucker Baseline, ADAM, and BFGS optimization methods.

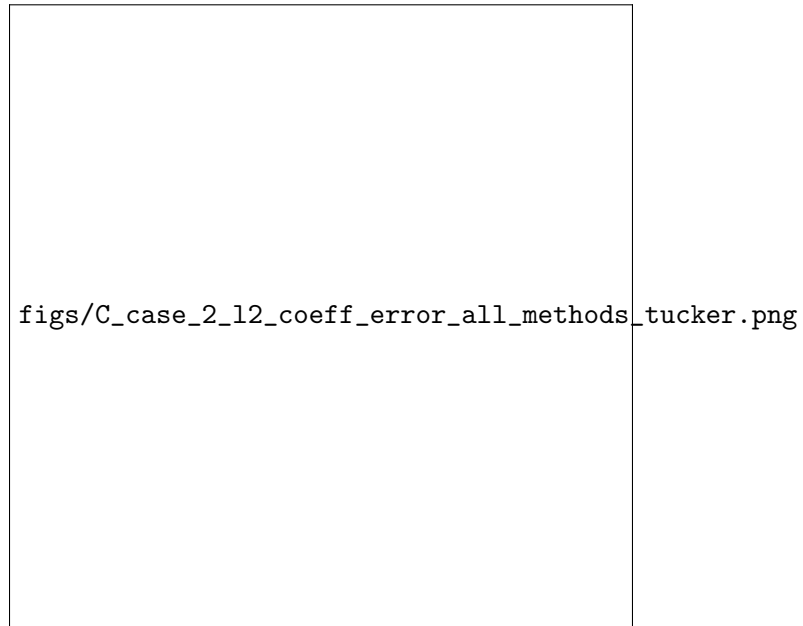


Figure 8: Case 2: Comparison of ALS Tucker, Tucker Baseline, ADAM, and BFGS optimization methods.

5 Comparison of Tucker Decomposition Methods

For a 2D function sampled on an $N_x \times N_y$ grid, we compare three Tucker-based approaches focusing on the mode-1 update:

Aspect	Classical Tucker	SVD-based ALS (Ours)	Direct ALS
Factorization	$F \approx U_x G U_y^T$ where $U_x \in \mathbb{R}^{N_x \times R_x}$, $U_y \in \mathbb{R}^{N_y \times R_y}$	$F \approx (T_x A_x) G (T_y A_y)^T$ where $A_x \in \mathbb{R}^{(d_x+1) \times R_x}$, $A_y \in \mathbb{R}^{(d_y+1) \times R_y}$	$F \approx (T_x A_x) G (T_y A_y)^T$ where $A_x \in \mathbb{R}^{(d_x+1) \times R_x}$, $A_y \in \mathbb{R}^{(d_y+1) \times R_y}$
Variables	Factor matrices U_x, U_y directly	Coefficient matrices A_x, A_y	Coefficient matrices A_x, A_y
Mode-1 update	$F \approx U_x G U_y^T$ $M = F U_y G^T$ $U_x = \text{SVD}_{R_x}(M)$	$F \approx T_x A_x G (T_y A_y)^T$ $\min \ F - T_x \tilde{G}_x (T_y A_y)^T\ _F$ $\tilde{G}_x = [(T_y A_y) \otimes T_x]^{\dagger} \text{vec}(F)$ $\tilde{G}_x = U^c S V^T$ (SVD) $A_x = U^c(:, 1: R_x), G = S(1: R_x, :) V^T$	$F \approx T_x A_x G (T_y A_y)^T$ $\min \ F - T_x A_x G (T_y A_y)^T\ _F$ $G_x = T_x^T T_x, H_y = A_y^T T_y^T T_y A_y$ $G_x A_x (G H_y G^T) = T_x^T F T_y A_y G^T$ $A_x = G_x^{-1} [T_x^T F T_y A_y G^T] [G H_y G^T]^{\dagger}$
d-dim extension (mode-n)	$U_n \leftarrow \text{SVD}_{R_n}(F_{(n)} U_{-n} G_{(n)}^T)$ where $U_{-n} = \bigotimes_{k \neq n} U_k$	$\tilde{G}_n = [U_{-n} \otimes T_n]^{\dagger} \text{vec}(F_{(n)})$ $\tilde{G}_n = U^c S V^T$ (SVD) $A_n = U^c(:, 1: R_n), G_{(n)} = S(1: R_n, :) V^T$	$G_n A_n M_n = T_n^T F_{(n)} (\bigotimes_{k \neq n} T_k A_k) G_{(n)}^T$ where $M_n = G_{(n)} (\bigotimes_{k \neq n} H_k) G_{(n)}^T$ $H_k = A_k^T (T_k^T T_k) A_k$ for $k \neq n$
Rank adaptation	Fixed rank R_x, R_y	Rank adapted via SVD truncation at each iteration	Fixed rank R_x, R_y
Parameters	$N_x R_x + N_y R_y + R_x R_y$	$(d_x + 1) R_x + (d_y + 1) R_y + R_x R_y$	$(d_x + 1) R_x + (d_y + 1) R_y + R_x R_y$
Interpretability	No functional meaning	Chebyshev coefficients	Chebyshev coefficients

Table 1: Comparison of three Tucker decomposition approaches for 2D functions. The key distinction is that classical Tucker operates directly on data factors U_x, U_y , while both ALS methods constrain factors to the Chebyshev basis span via $U = T A$. The SVD-based ALS additionally adapts the core tensor rank during optimization.

5.1 Detailed Method Comparison

5.1.1 Classical Tucker Decomposition

Classical Tucker decomposition directly factorizes the data tensor $F \in \mathbb{R}^{N_x \times N_y}$ as

$$F \approx U_x G U_y^T,$$

where $U_x \in \mathbb{R}^{N_x \times R_x}$ and $U_y \in \mathbb{R}^{N_y \times R_y}$ are arbitrary orthogonal factor matrices, and $G \in \mathbb{R}^{R_x \times R_y}$ is the core tensor.

Mode-1 ALS Update: The goal is to update U_x while holding U_y and G fixed. We minimize

$$\min_{U_x} \|F - U_x G U_y^T\|_F.$$

The solution involves computing the product

$$M = F U_y G^T \in \mathbb{R}^{N_x \times R_x},$$

which represents the optimal U_x in the full column space. To enforce orthogonality and extract the rank- R_x approximation, we apply SVD truncation:

$$M = USV^T, \quad \text{then} \quad U_x = U(:, 1 : R_x).$$

This is the classical higher-order SVD (HOSVD) approach applied iteratively within ALS.

Characteristics:

- Variables: $N_x R_x + N_y R_y + R_x R_y$ parameters
- No structural constraint on factors — U_x, U_y can be any orthogonal matrices
- No functional interpretation of U_x, U_y — purely data-driven decomposition
- Cannot exploit function smoothness or leverage any known basis structure
- Suitable for general tensor approximation but not tailored to spectral methods

5.1.2 SVD-Based ALS (Our Method)

Our method constrains factors to lie in the span of Chebyshev basis functions. Given basis matrices $T_x \in \mathbb{R}^{N_x \times (d_x+1)}$ and $T_y \in \mathbb{R}^{N_y \times (d_y+1)}$ evaluated at the sample points, we seek

$$F \approx (T_x A_x) G (T_y A_y)^T,$$

where $A_x \in \mathbb{R}^{(d_x+1) \times R_x}$ and $A_y \in \mathbb{R}^{(d_y+1) \times R_y}$ are coefficient matrices.

Mode-1 ALS Update: Fix A_y and G . Define $U_y = T_y A_y \in \mathbb{R}^{N_y \times R_y}$, which embeds the current mode-2 coefficients into sample space. The constrained optimization problem is

$$\min_{\tilde{G}_x} \|F - T_x \tilde{G}_x U_y^T\|_F,$$

where $\tilde{G}_x \in \mathbb{R}^{(d_x+1) \times R_y}$ represents an augmented coefficient matrix that simultaneously encodes both A_x and the core G . This is a standard linear least-squares problem. Vectorizing both sides:

$$\text{vec}(F) \approx (U_y \otimes T_x) \text{vec}(\tilde{G}_x),$$

where $\otimes$ denotes the Kronecker product. The least-squares solution is

$$\text{vec}(\tilde{G}_x) = (U_y \otimes T_x)^\dagger \text{vec}(F),$$

which can be reshaped to yield $\tilde{G}_x \in \mathbb{R}^{(d_x+1) \times R_y}$.

To separate the mode-1 coefficients A_x from the updated core tensor G , we apply SVD:

$$\tilde{G}_x = U^c S V^T.$$

Truncating to rank R_x , we extract

$$A_x = U^c(:, 1 : R_x) \in \mathbb{R}^{(d_x+1) \times R_x},$$

and form the updated mode-1 unfolding of the core as

$$G_{(1)} = S(1 : R_x, 1 : R_x) V^T(1 : R_x, :) \in \mathbb{R}^{R_x \times R_y},$$

which is then reshaped back into the 2D core tensor G .

Characteristics:

- Variables: $(d_x + 1)R_x + (d_y + 1)R_y + R_x R_y$ parameters (typically $d_x \ll N_x$)
- Factors A_x, A_y represent Chebyshev coefficients with clear spectral meaning
- Rank adaptation: SVD truncation allows dynamic rank adjustment at each iteration
- Exploits smoothness through basis representation — smooth functions require fewer coefficients
- Handles over-sampled data ($N > d + 1$) naturally via least-squares projection
- Works on arbitrary sample points (not restricted to structured grids)

5.1.3 Direct ALS (Fixed Rank)

The Direct ALS method also constrains factors to the Chebyshev basis span but solves the normal equations directly without SVD, maintaining fixed rank throughout.

Mode-1 ALS Update: Fix $A_y \in \mathbb{R}^{(d_y+1) \times R_y}$ and $G \in \mathbb{R}^{R_x \times R_y}$. The optimization problem is

$$\min_{A_x} \|F - T_x A_x G (T_y A_y)^T\|_F.$$

To solve this, we work with the normal equations. Define the Gram matrices:

$$G_x = T_x^T T_x \in \mathbb{R}^{(d_x+1) \times (d_x+1)}, \quad H_y = A_y^T (T_y^T T_y) A_y \in \mathbb{R}^{R_y \times R_y}.$$

Here, G_x is the Gram matrix of the Chebyshev basis (constant for all iterations), and H_y encodes the metric induced by the mode-2 coefficients.

Taking the derivative with respect to A_x and setting it to zero yields the normal equations:

$$G_x A_x (G H_y G^T) = T_x^T F T_y A_y G^T.$$

The left side has the structure of a generalized Sylvester equation. Solving for A_x explicitly:

$$A_x = G_x^{-1} [T_x^T F T_y A_y G^T] [G H_y G^T]^\dagger,$$

where G_x^{-1} exists since the Chebyshev Vandermonde matrix has full column rank, and $[G H_y G^T]^\dagger$ denotes the Moore-Penrose pseudo-inverse to handle potential rank deficiencies in the core-metric product.

Characteristics:

- Variables: $(d_x + 1)R_x + (d_y + 1)R_y + R_x R_y$ parameters (same as SVD-based ALS)
- Factors A_x, A_y represent Chebyshev coefficients
- No rank adaptation: ranks R_x, R_y remain fixed throughout all iterations
- Avoids SVD computation — potentially faster per iteration but no automatic rank selection
- Requires matrix pseudo-inverse $[G H_y G^T]^\dagger$ which may become ill-conditioned
- Gram matrix G_x can be precomputed once and reused across all iterations
- Best suited when the optimal ranks are known a priori

5.1.4 Extension to Higher Dimensions

For 3D problems with $F \in \mathbb{R}^{N_x \times N_y \times N_z}$, the Direct ALS mode-1 update becomes:

$$G_x A_x M_1 = T_x^T F_{(1)} (T_z A_z \otimes T_y A_y) G_{(1)}^T,$$

where $F_{(1)} \in \mathbb{R}^{N_x \times (N_y N_z)}$ is the mode-1 unfolding of F , and

$$M_1 = G_{(1)} (H_z \otimes H_y) G_{(1)}^T \in \mathbb{R}^{R_x \times R_x},$$

with $H_y = A_y^T (T_y^T T_y) A_y$ and $H_z = A_z^T (T_z^T T_z) A_z$.

Computational Scaling: The Kronecker products grow exponentially with dimension d . For arbitrary dimension d , updating mode- n involves computing:

- $\bigotimes_{k \neq n} T_k A_k$: scales as $\mathcal{O}(N^{d-1} R)$, where N is typical grid size and R is typical rank
- $\bigotimes_{k \neq n} H_k$: scales as $\mathcal{O}(R^{d-1})$

The SVD-based method has similar Kronecker product costs but additionally performs SVD on the augmented coefficient matrix of size $(d_n + 1) \times (R^{d-1})$, costing $\mathcal{O}((d_n + 1) R^{2(d-1)})$. This provides rank adaptivity at extra computational expense, which is often worthwhile when the optimal rank is not known in advance.